

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM306 Параллел ба хуваарилагдсан тооцоолол

Нэгтгэх эрэмбэлэлт

БИЕ ДААЛТ

Шалгасан багш:

Алгоритмын дугаар:

Гүйцэтгэсэн оюутан:

Н.Анхбаяр /F.CS20/

№6

О.Дэлгэрмаа /B232270024/

Т.Мөнхцэцэн /B232270025/

Б.Цогжавхлан /B222270838/

И.Эрдэнэбаяр /B232270136/

Улаанбаатар хот

2026 он

Гарчиг

1	Хураангуй	1
2	Оршил	2
3	Онолын үндэс	3
3.1	Merge Sort алгоритм	3
3.2	Параллел тооцооллын үндэс	3
3.3	CPU олон цөм ба <code>std::thread</code>	3
3.4	OpenMP	4
3.5	CUDA ба GPU архитектур	4
4	Алгоритмын зохиомж	5
4.1	Дараалсан хувилбарын зохиомж	5
4.1.1	Дараалсан хувилбарын псевдо код	5
4.2	<code>std::thread</code> хувилбарын зохиомж	6
4.2.1	<code>std::thread</code> хувилбарын псевдо код	7
4.3	OpenMP хувилбарын зохиомж	7
4.3.1	OpenMP хувилбарын псевдо код	7
4.4	CUDA хувилбарын зохиомж	8
4.4.1	CUDA хувилбарын псевдо код	9
5	Туршилтын орчин ба хэмжилтийн аргачлал	10
5.1	Туршилтын орчин	10
5.2	Өгөгдлийн хэмжээ	10
5.3	Зөв ажиллагааны шалгалт	10
5.4	Хэмжилтийн үзүүлэлтүүд	11
5.4.1	Execution time	11
5.4.2	Speedup	11
5.4.3	Total operations	11
5.4.4	Achievable performance	11
5.4.5	CUDA өгөгдөл дамжуулалт	11
6	Үр дүн ба харьцуулсан шинжилгээ	12
6.1	Execution time харьцуулалт	12
6.2	Speedup харьцуулалт	12
6.3	CUDA хугацааны задаргаа	13
6.4	Total operations ба achievable performance	14
6.5	Харьцуулсан шинжилгээ	15

7	Санах ой, кэш, регистрийн түвшний тайлбар	16
7.1	CPU санах ой ба кэшийн түвшин	16
7.2	Thread түвшний ажиллагаа	16

1. Хураангуй

Энэхүү бие даалтын ажлын хүрээнд нэгтгэх эрэмбэлэлт буюу Merge Sort алгоритмыг нийт дөрвөн өөр хувилбараар хэрэгжүүлж, тэдгээрийн гүйцэтгэлийг харьцуулан шинжилсэн. Үүнд энгийн дараалсан хувилбар, C++ стандарт сангийн `std::thread` ашигласан CPU олон цөмт хувилбар, OpenMP compiler directive ашигласан CPU параллел хувилбар, мөн CUDA ашигласан GPU-д суурилсан хувилбар багтсан.

Туршилтыг 10,000, 100,000, 1,000,000 ширхэг бүхэл тоон өгөгдөл дээр гүйцэтгэж, хувилбар бүрийн execution time, speedup, total operations, achievable performance, CUDA хувилбарын хувьд Host-to-Device, Kernel execution, Device-to-Host data transfer time зэрэг үзүүлэлтүүдийг хэмжсэн. Бүх хувилбаруудын зөв ажиллагааг C++ стандарт сангийн `std::sort` функцийн үр дүнтэй харьцуулан шалгасан.

Бие даалтын зорилго нь ижил алгоритмыг өөр өөр параллел тооцооллын технологиор хэрэгжүүлэхэд гүйцэтгэл хэрхэн өөрчлөгдөхийг тодорхойлох, мөн CPU олон цөм, OpenMP task-based parallelism, GPU thread-block-grid архитектурын ялгааг практик туршилтаар харуулахад оршино.

2. Оршил

Орчин үеийн тооцооллын системүүд нь нэг цөмт CPU дээр тулгуурласан дараалсан боловсруулалтаас олон цөмт CPU болон олон мянган урсгал зэрэг ажиллуулах чадвартай GPU архитектур руу шилжсэн. Иймээс алгоритмыг зөвхөн зөв ажиллуулах бус, тухайн архитектурын онцлогт нийцүүлэн параллел хэлбэрээр зохион байгуулах шаардлага улам бүр нэмэгдэж байна.

Эрэмбэлэлтийн алгоритмууд нь өгөгдөл боловсруулалт, хайлт, өгөгдлийн сан, график боловсруулалт, хиймэл оюун ухаан зэрэг олон салбарт өргөн хэрэглэгддэг. Тэдгээрийн дундаас Merge Sort нь divide-and-conquer зарчимд суурилдаг тул параллелчлэхад тохиромжтой алгоритмуудын нэг юм. Учир нь массивыг хоёр хэсэгт хуваасны дараа зүүн болон баруун дэд массивуудыг хоорондоо хамааралгүйгээр зэрэг эрэмбэлэх боломжтой.

Энэхүү ажлын хүрээнд Merge Sort алгоритмыг дараах дөрвөн хувилбараар хэрэгжүүлсэн.

1. Дараалсан Merge Sort.
2. `std::thread` ашигласан CPU олон цөмт Merge Sort.
3. OpenMP task directive ашигласан Merge Sort.
4. CUDA ашигласан GPU bottom-up Merge Sort.

Гүйцэтгэлийг харьцуулахдаа дараах үндсэн үзүүлэлтүүдийг ашигласан.

- Execution time буюу ажиллах хугацаа.
- Speedup буюу дараалсан хувилбартай харьцуулсан хурдатгал.
- Total operations буюу ойролцоолсон үйлдлийн тоо.
- Achievable performance буюу нэг секундэд гүйцэтгэсэн сая үйлдлийн тоо.
- CUDA хувилбарын өгөгдөл дамжуулах хугацаа.

3. Онолын үндэс

3.1 Merge Sort алгоритм

Merge Sort нь divide-and-conquer буюу хуваа, шийд, нэгтгэ зарчимд суурилсан эрэмбэлэлтийн алгоритм юм. Алгоритм нь өгөгдсөн массивыг хоёр хэсэгт хувааж, хэсэг бүрийг дахин рекурсивээр эрэмбэлээд, дараа нь хоёр эрэмбэлэгдсэн хэсгийг нэгтгэдэг.

Merge Sort алгоритмын ерөнхий ажиллагаа дараах гурван шаттай.

1. **Divide:** Массивыг зүүн болон баруун хоёр дэд массив болгон хуваана.
2. **Conquer:** Хоёр дэд массивыг тус тусад нь эрэмбэлнэ.
3. **Merge:** Эрэмбэлэгдсэн хоёр дэд массивыг нэг эрэмбэлэгдсэн массив болгон нэгтгэнэ.

Merge Sort-ийн хугацааны төвөгшил нь хамгийн сайн, дундаж, хамгийн муу бүх тохиолдолд[1]:

$$T(n) = O(n \log_2 n) \quad (3.1)$$

Санах ойн нэмэлт хэрэглээ нь ихэвчлэн:

$$S(n) = O(n) \quad (3.2)$$

байдаг. Учир нь merge хийх үед түр массивууд ашиглагддаг.

3.2 Параллел тооцооллын үндэс

Параллел тооцоолол нь нэг том бодлогыг хэд хэдэн жижиг ажлуудад хувааж, тэдгээрийг зэрэг ажиллуулах замаар нийт гүйцэтгэх хугацааг багасгах арга юм. Merge Sort нь зүүн болон баруун дэд массивуудыг зэрэг эрэмбэлэх боломжтой тул параллел тооцоололд тохиромжтой.

Параллел алгоритмын үр ашгийг ихэвчлэн speedup үзүүлэлтээр хэмждэг.

$$Speedup = \frac{T_{sequential}}{T_{parallel}} \quad (3.3)$$

Энд $T_{sequential}$ нь дараалсан хувилбарын хугацаа, $T_{parallel}$ нь параллел хувилбарын хугацаа юм.

3.3 CPU олон цөм ба `std::thread`

`std::thread` нь C++ стандарт сангийн урсгал үүсгэх, ажиллуулах, хүлээх боломж олгодог хэрэгсэл юм. Merge Sort-ийн хувьд массивын зүүн хэсгийг нэг урсгалд, баруун хэсгийг үндсэн урсгал эсвэл өөр урсгалд өгч зэрэг ажиллуулж болно.

Thread ашиглах үед дараах зүйлсийг анхаарах шаардлагатай.

- Thread үүсгэх нь өөрөө зардалтай үйлдэл.
- Хэт олон thread үүсгэвэл context switching нэмэгдэнэ.
- Merge хийхээс өмнө хоёр дэд хэсэг хоёулаа эрэмбэлэгдсэн байх ёстой.
- Үүнийг `join()` ашиглан синхрончилно.

3.4 OpenMP

OpenMP нь C/C++ болон Fortran хэлэнд compiler directive ашиглан олон thread дээр параллел тооцоолол хийх боломж олгодог ба task-based parallelism-ийг дэмждэг [2]. OpenMP-ийн давуу тал нь thread pool, task scheduling, синхрончлол зэрэг ажлыг хөгжүүлэгч бага кодоор илэрхийлэх боломжтой байдаг.

Энэхүү ажилд OpenMP-ийн дараах directive-үүдийг ашигласан.

- `#pragma omp parallel`: OpenMP thread pool үүсгэнэ.
- `#pragma omp single`: Рекурсив task үүсгэх ажлыг нэг урсгал эхлүүлнэ.
- `#pragma omp task`: Зүүн болон баруун дэд бодлогуудыг task болгон үүсгэнэ.
- `#pragma omp taskwait`: Хоёр task дууссаны дараа merge хийхийг баталгаажуулна.

Жижиг хэмжээний дэд массив дээр task үүсгэх overhead нь бодит эрэмбэлэх ажлаас их болох боломжтой. Иймээс `OMP_CUTOFF` утга ашиглаж, тодорхой хэмжээнээс бага массивыг дараалсан Merge Sort-р эрэмбэлсэн.

3.5 CUDA ба GPU архитектур

CUDA нь NVIDIA GPU дээр параллел программ ажиллуулах программчлалын платформ юм. Энэ нь GPU дээр параллел тооцоолол хийх боломж олгодог ба grid, block, thread гэсэн шаталсан бүтэцтэй [3].

CUDA программын үндсэн санах ойн урсгал дараах байдалтай.

1. Host буюу CPU санах ойд өгөгдөл үүсгэнэ.
2. `cudaMemcpy` ашиглан өгөгдлийг Device буюу GPU санах ой руу хуулна.
3. CUDA kernel ажиллаж өгөгдлийг боловсруулна.
4. Үр дүнг GPU санах ойгоос CPU санах ой руу буцааж хуулна.

Энэ ажлын CUDA хувилбар нь bottom-up Merge Sort хэлбэрээр хэрэгжсэн. Учир нь GPU дээр олон жижиг merge үйлдлийг нэгэн зэрэг kernel thread-үүдээр ажиллуулах нь илүү тохиромжтой.

4. Алгоритмын зохиомж



Зураг 4.1: Merge Sort алгоритмын ерөнхий ажиллагаа

4.1 Дараалсан хувилбарын зохиомж

Дараалсан нэгтгэх эрэмбэлэлт хувилбар нь нэг CPU урсгал дээр ажиллана. Массивыг рекурсивээр хоёр хэсэгт хувааж, дэд массив бүрийг дараалсан байдлаар эрэмбэлээд нэгтгэдэг.

4.1.1 Дараалсан хувилбарын псевдо код

```
1 MERGE_PARTS(A, left, mid, right):
2     L = copy A[left ... mid]
3     R = copy A[mid + 1 ... right]
4
5     i = 0
6     j = 0
7     k = left
8
9     while i < length(L) and j < length(R):
10         if L[i] <= R[j]:
11             A[k] = L[i]
12             i = i + 1
13         else:
14             A[k] = R[j]
15             j = j + 1
16         k = k + 1
17
```



```

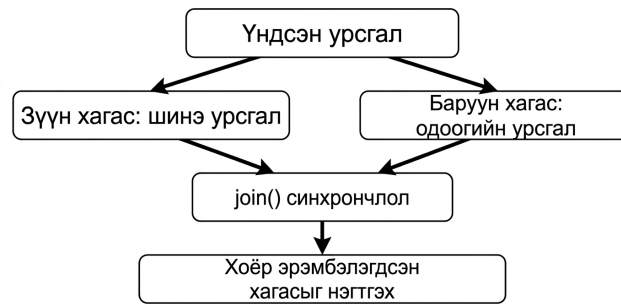
18     while i < length(L):
19         A[k] = L[i]
20         i = i + 1
21         k = k + 1
22
23     while j < length(R):
24         A[k] = R[j]
25         j = j + 1
26         k = k + 1
27
28 MERGE_SORT_SEQUENTIAL(A, left, right):
29     if left >= right:
30         return
31
32     mid = left + (right - left) / 2
33
34     MERGE_SORT_SEQUENTIAL(A, left, mid)
35     MERGE_SORT_SEQUENTIAL(A, mid + 1, right)
36
37     MERGE_PARTS(A, left, mid, right)
38
39 RUN_SEQUENTIAL(A):
40     if A is empty:
41         return 0
42
43     startTime = current time
44     MERGE_SORT_SEQUENTIAL(A, 0, length(A) - 1)
45     endTime = current time
46     return endTime - startTime
47
48 BENCHMARK_ONE(n):
49     A = MAKE_RANDOM_DATA(n)
50     expected = copy A
51     STD_SORT(expected)
52     ms = RUN_SEQUENTIAL(A)
53     correct = (A == expected)
54     SAVE_TO_CSV("Sequential", n, ms, correct)

```

4.2 std::thread хувилбарын зохиомж

`std::thread` хувилбарт нэгтгэх эрэмбэлэлтийн зүүн дэд массивыг шинэ урсгал дээр ажиллуулж, баруун дэд массивыг үндсэн урсгал дээр ажиллуулсан. Хоёр талын эрэмбэлэлт дууссаны дараа `join()` ашиглан синхрончлол хийж, дараа нь `merge` үйлдлийг гүйцэтгэнэ.

Урсгалуудын тоог хэт их болгохгүйн тулд `maxDepth` утга ашигласан. Хэрэв `depth < maxDepth` бол шинэ урсгал үүсгэнэ, эсрэг тохиолдолд дараалсан нэгтгэх эрэмбэлэлт шилжинэ.



Зураг 4.2: std::thread ашигласан Merge Sort-ийн ажиллагаа

4.2.1 std::thread хувилбарын псевдо код

```

1 MERGE_SORT_THREADED(A, left, right, depth, maxDepth):
2     if left >= right:
3         return
4
5     mid = (left + right) / 2
6
7     if depth < maxDepth:
8         spawn thread:
9             MERGE_SORT_THREADED(A, left, mid, depth+1, maxDepth)
10
11             MERGE_SORT_THREADED(A, mid+1, right, depth+1, maxDepth)
12
13         join thread
14     else:
15         MERGE_SORT_SEQ(A, left, mid)
16         MERGE_SORT_SEQ(A, mid+1, right)
17
18     MERGE_PARTS(A, left, mid, right)
  
```

4.3 OpenMP хувилбарын зохиомж

OpenMP хувилбарт дахин дуудагдах нэгтгэх эрэмбэлэлтийн хоёр дэд дуудлагыг `#pragma omp task directive` ашиглан task болгон үүсгэсэн. OpenMP runtime нь эдгээр task-уудыг thread pool доторх боломжит урсгалуудад хуваарилна.

Merge хийхээс өмнө зүүн болон баруун хэсэг хоёулаа бүрэн эрэмбэлэгдсэн байх ёстой. Иймээс `#pragma omp taskwait directive` ашиглан синхрончлол хийсэн.

4.3.1 OpenMP хувилбарын псевдо код

```

1 MERGE_SORT_OPENMP(A, left, right):
2     if left >= right:
3         return
4
  
```

```

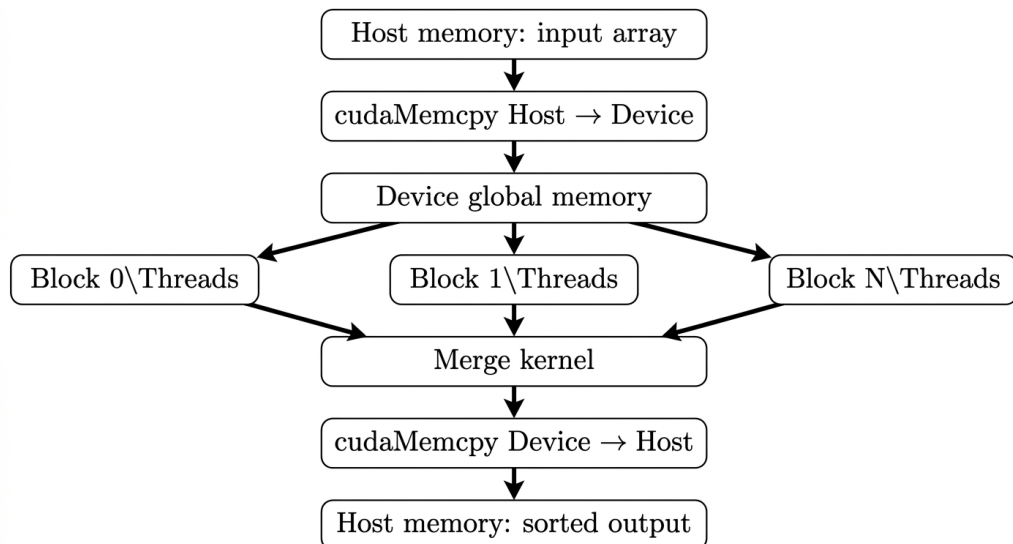
5     if size(A) <= CUTOFF:
6         MERGE_SORT_SEQ(A, left, right)
7         return
8
9     mid = (left + right) / 2
10
11     #pragma omp task
12         MERGE_SORT_OPENMP(A, left, mid)
13
14     #pragma omp task
15         MERGE_SORT_OPENMP(A, mid+1, right)
16
17     #pragma omp taskwait
18
19     MERGE_PARTS(A, left, mid, right)

```

4.4 CUDA хувилбарын зохиомж

CUDA хувилбарт доороос дээш нэгтгэх эрэмбэлэлт ашигласан. Дахин дуудагдах нэгтгэх эрэмбэлэлт нь GPU дээр шууд хэрэгжүүлэхэд тохиромж багатай тул merge хийх шат бүрийг width утгаар удирдсан. Эхлээд width = 1 үед хоёр нэгж элементүүдийг нэгтгэнэ. Дараа нь width = 2, 4, 8 гэх мэтээр хоёр дахин өсгөж, бүх массив эрэмбэлэгдэх хүртэл kernel ажиллуулна.

CUDA kernel дээр нэг урсгал нь нэг merge segment боловсруулах байдлаар зохион байгуулагдсан. Урсгалууд block дотор, block-ууд grid дотор байрлана.



Зураг 4.3: CUDA хувилбарын санах ой ба grid/block/thread бүтэц

4.4.1 CUDA хувилбарын псевдо код

```
1  RUN_CUDA(A):
2      expected = copy A
3      STD_SORT(expected)
4
5      allocate GPU memory (d_arr, d_tmp)
6
7      start H2D timer
8      copy A -> GPU
9      stop H2D timer
10
11     start kernel timer
12     MERGE_SORT_CUDA(d_arr)
13     stop kernel timer
14
15     start D2H timer
16     copy GPU -> A
17     stop D2H timer
18
19     correct = (A == expected)
20
21     totalTime = H2D + kernel + D2H
22
23     SAVE_TO_CSV(
24         algorithm = "CUDA",
25         executionTime = kernel,
26         totalTime = totalTime,
27         correct = correct
28     )
```

5. Туршилтын орчин ба хэмжилтийн аргачлал

5.1 Туршилтын орчин

Туршилтыг дараах орчинд гүйцэтгэнэ. Доорх хүснэгтэд өөрийн компьютерийн бодит үзүүлэлтүүдийг бөглөнө.

Хүснэгт 5.1: Туршилтын орчны мэдээлэл

Үзүүлэлт	Утга
Үйлдлийн систем	Windows 10 / Windows 11
CPU	11th Gen Intel Core i5-11400H @ 2.70GHz
CPU цөмийн тоо	6 cores / 12 logical processors
RAM	16 GB
GPU	NVIDIA GeForce RTX 3050 Laptop GPU
CUDA version	12.5
C++ compiler	g++ / MSYS2 UCRT64 / MSVC
OpenMP compiler flag	-fopenmp
CUDA compiler	nvcc

5.2 Өгөгдлийн хэмжээ

Туршилтад дараах гурван хэмжээтэй санамсаргүй бүхэл тоон массив ашигласан.

- 10,000 элемент.
- 100,000 элемент.
- 1,000,000 элемент.

Бүх хувилбарт ижил өгөгдөл ашиглахын тулд deterministic random generator хэрэглэсэн. Ингэснээр algorithm бүр ижил input дээр ажиллаж, харьцуулалт шударга болно.

5.3 Зөв ажиллагааны шалгалт

Хувилбар бүрийн үр дүнг C++ стандарт сангийн `std::sort` функцээр эрэмбэлсэн expected массивтай харьцуулсан. Хэрэв хоёр массив ижил байвал тухайн хувилбарыг зөв ажилласан гэж үзсэн.

5.4 Хэмжилтийн үзүүлэлтүүд

5.4.1 Execution time

CPU хувилбаруудын хувьд execution time нь тухайн algorithm ажилласан нийт хугацаа юм. CUDA хувилбарын хувьд нийт хугацааг Host-to-Device copy, kernel execution, Device-to-Host copy хугацааны нийлбэрээр авч үзсэн.

$$T_{CUDA,total} = T_{H2D} + T_{kernel} + T_{D2H} \quad (5.1)$$

5.4.2 Speedup

Speedup нь дараалсан хувилбарын хугацааг тухайн параллел хувилбарын нийт хугацаанд хувааж бодсон.

$$Speedup = \frac{T_{sequential}}{T_{algorithm}} \quad (5.2)$$

5.4.3 Total operations

Merge Sort-ийн ойролцоолсон нийт үйлдлийн тоог дараах байдлаар тооцсон.

$$Operations \approx n \log_2 n \quad (5.3)$$

5.4.4 Achievable performance

Achievable performance буюу MOPS үзүүлэлтийг нэг секундэд гүйцэтгэх сая үйлдлийн тоогоор илэрхийлсэн.

$$MOPS = \frac{Operations}{T_{seconds} \times 10^6} \quad (5.4)$$

5.4.5 CUDA өгөгдөл дамжуулалт

CUDA хувилбарт CPU санах ойгоос GPU санах ой руу, мөн GPU санах ойгоос CPU санах ой руу өгөгдөл хуулсан. Нийт дамжуулсан өгөгдлийн хэмжээг дараах байдлаар тооцсон.

$$TransferredBytes = 2 \times n \times sizeof(int) \quad (5.5)$$

Энд 2 гэдэг нь Host-to-Device болон Device-to-Host гэсэн хоёр чиглэлийн дамжуулалтыг илэрхийлнэ.

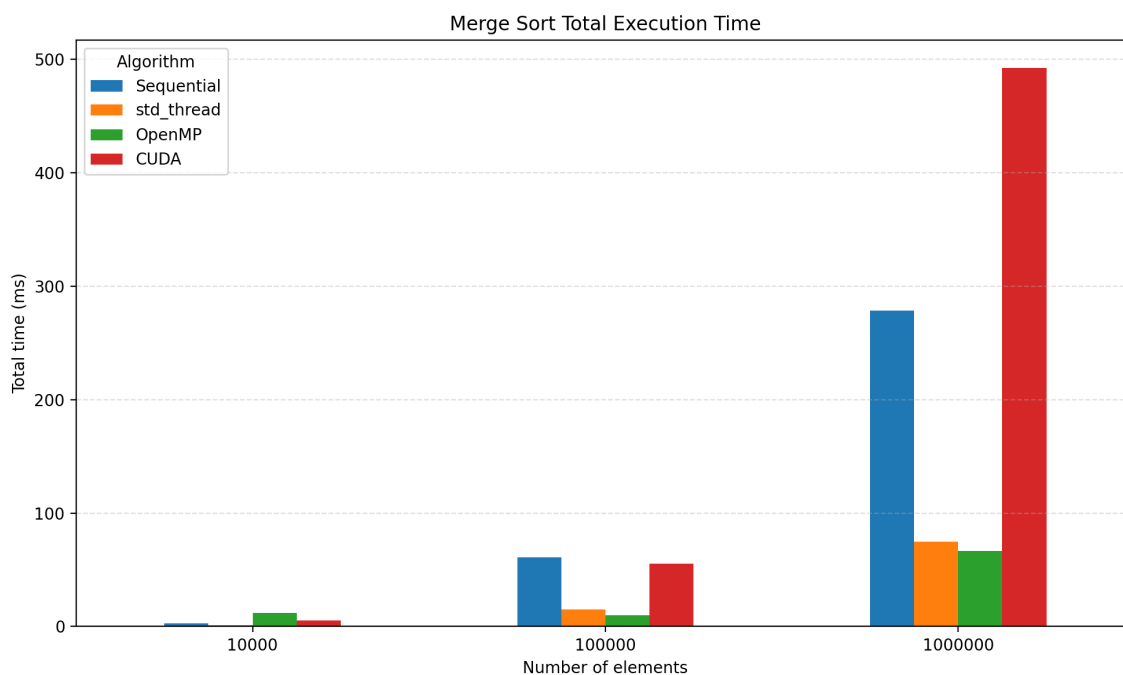
6. Үр дүн ба харьцуулсан шинжилгээ

6.1 Execution time харьцуулалт

Доорх хүснэгтэд algorithm бүрийн 10,000, 100,000, 1,000,000 элемент дээр ажилласан нийт хугацааг харуулна. Утгуудыг `results.csv` файлаас авна.

Хүснэгт 6.1: Execution time харьцуулалт

Algorithm	10k ms	100k ms	1M ms
Sequential	2.731	60.865	278.438
std::thread	1.215	15.022	74.612
OpenMP	12.065	10.162	66.695
CUDA	5.243	55.539	492.434



Зураг 6.1: Execution time график

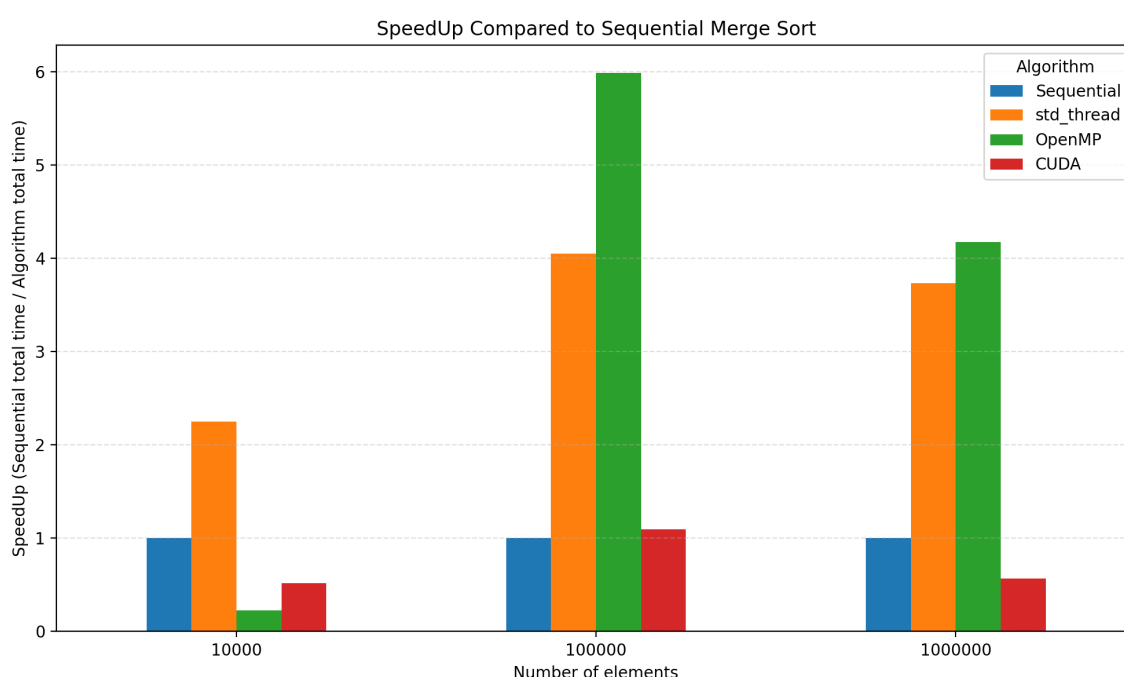
6.2 Speedup харьцуулалт

Speedup нь дараалсан хувилбартай харьцуулсан хурдатгал юм. Хэрэв speedup 1-ээс их байвал тухайн хувилбар дараалсан хувилбараас хурдан ажилласан гэсэн үг.

Хүснэгт 6.2: Speedup харьцуулалт

Algorithm	10k	100k	1M
Sequential	1.00	1.00	1.00
std::thread	2.25	4.05	3.73
OpenMP	0.23	5.99	4.18
CUDA	0.52	1.10	0.57

Энэхүү хэрэгжүүлэлтийн CUDA хувилбар 1,000,000 элемент дээр CPU parallel хувилбаруудаас удаан ажилласан. Үүний гол шалтгааныг bottom-up merge sort kernel-ийн global memory дээр олон удаа уншиж бичих ажиллагаа болон kernel-ийн үе шат бүрийн синхрончлолын зардалтай холбоотой гэж үзэж байгаа.



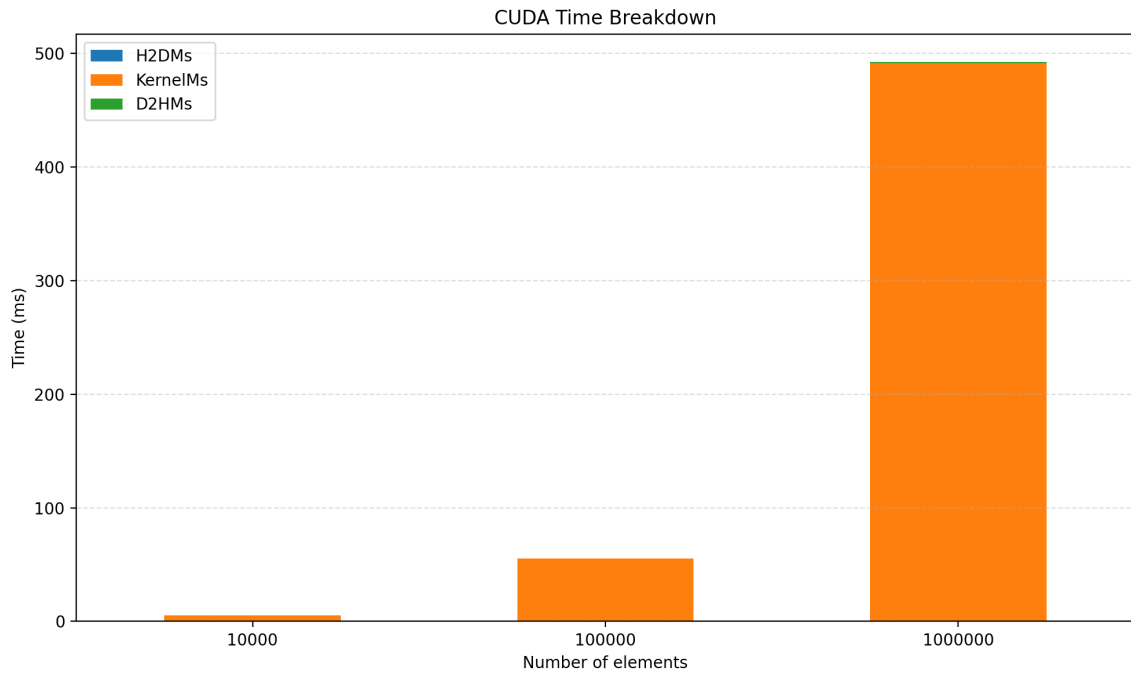
Зураг 6.2: Speedup харьцуулалтын график

6.3 CUDA хугацааны задаргаа

CUDA хувилбарын хувьд нийт хугацаа нь өгөгдөл дамжуулах болон kernel ажиллах хугацаанаас бүрдэнэ. Туршилтын үр дүнгээс харахад Host-to-Device болон Device-to-Host дамжуулалтын хугацаа харьцангуй бага байсан бол kernel execution хугацаа нийт хугацааны ихэнхийг эзэлсэн. Иймээс энэхүү хэрэгжүүлэлт дээр CUDA-ийн удаашрал нь data transfer-аас илүү kernel доторх merge үйлдэл, global memory access, шат бүрийн синхрончлолтой холбоотой байна.

Хүснэгт 6.3: CUDA хугацааны задаргаа

N	H2D ms	Kernel ms	D2H ms	Total ms
10,000	0.035840	5.170176	0.037184	5.243200
100,000	0.124736	55.254593	0.159392	55.538721
1,000,000	0.909248	490.475800	1.048960	492.434008



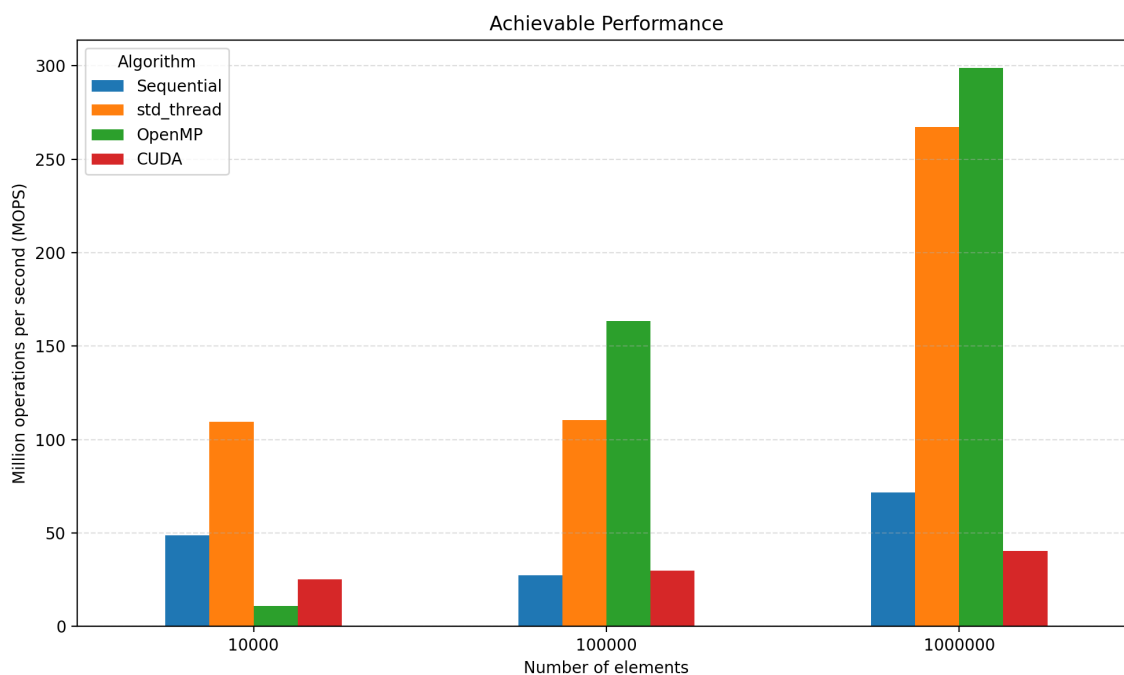
Зураг 6.3: CUDA H2D, Kernel, D2H хугацааны задаргаа

6.4 Total operations ба achievable performance

Merge Sort-ийн нийт үйлдлийн тоог $n \log_2 n$ гэж ойролцоолсон. Үүний үндсэн дээр MOPS буюу нэг секундэд гүйцэтгэсэн сая үйлдлийн тоог тооцсон.

Хүснэгт 6.4: Ойролцоолсон total operations

N	Estimated operations
10,000	132,877
100,000	1,660,964
1,000,000	19,931,569



Зураг 6.4: Achievable performance буюу MOPS харьцуулалт

6.5 Харьцуулсан шинжилгээ

Туршилтын үр дүнгээс дараах ажиглалтууд гарч байна.

- 10,000 элемент дээр std::thread хувилбар хамгийн хурдан (1.215 ms) ажилласан бол OpenMP болон CUDA хувилбаруудын хувьд parallelization overhead илүү нөлөөлсөн байна.
- 100,000 элемент дээр OpenMP (10.162 ms) болон std::thread (15.022 ms) хувилбарууд дараалсан хувилбараас (60.865 ms) мэдэгдэхүйц хурдан ажилласан.
- 1,000,000 элемент дээр OpenMP (66.695 ms) болон std::thread (74.612 ms) хувилбарууд хамгийн сайн гүйцэтгэл үзүүлсэн бөгөөд дараалсан хувилбараас 3-4 дахин хурдан болсон.
- CUDA хувилбар 1,000,000 элемент дээр 492.434 ms ажилласан нь бусад бүх хувилбараас удаан байгааг харуулж байна.
- Энэ нь CUDA хувилбар зөв ажилласан хэдий ч merge sort алгоритмын memory-intensive шинж, global memory access ихтэй байдал, мөн kernel үе шат бүрийн синхрончлолын зардалтай холбоотой.
- Иймээс GPU ашигласан хувилбар нь зөвхөн thread ихтэйгээр хурдан болно гэж үзэх нь буруу бөгөөд алгоритмын онцлогоос хамаарч гүйцэтгэл ялгаатай гарч болохыг харуулж байна.

7. Санах ой, кэш, регистрийн түвшний тайлбар

7.1 CPU санах ой ба кэшийн түвшин

CPU дээр ажиллах sequential, `std::thread`, OpenMP хувилбарууд нь үндсэндээ RAM дахь массив дээр ажиллана. Merge хийх үед зүүн болон баруун хэсгийг түр `vector` массив руу хуулж байгаа тул нэмэлт санах ой ашиглагдана.

CPU нь өгөгдөлд хандахдаа L1, L2, L3 cache ашигладаг. Дараалсан массивын элементүүд дараалан уншигдах үед cache locality сайн байдаг. Гэхдээ merge хийх үед түр массив үүсгэх, өгөгдлийг хуулбарлах үйлдэл их хийгддэг тул memory bandwidth нь гүйцэтгэлд нөлөөлж болно.

7.2 Thread түвшний ажиллагаа

`std::thread` хувилбарт thread бүр өөр дэд массив дээр ажиллаж байгаа тул өгөгдлийн давхцал бага. Гэхдээ merge хийх үе шатанд parent thread хоёр талын үр дүнг нэгтгэх хүртэл хүлээх шаардлагатай. Энэ синхрончлол нь `join()` функцээр хийгдэнэ.

OpenMP хувилбарт thread-үүдийг programmer шууд үүсгэхгүй, харин OpenMP runtime task-уудыг thread pool дээр хуваарилна. Энэ нь thread management-ийн зардлыг багасгаж, кодыг цэвэр болгодог.

7.3 GPU санах ой ба регистрийн түвшин

CUDA хувилбарт өгөгдөл CPU-ийн host memory-оос GPU-ийн global memory руу хуулж байрлана. Kernel ажиллах үед thread бүр өөрийн register-д индексүүд болох `i`, `j`, `k`, `left`, `mid`, `right` зэрэг утгуудыг хадгалж ажиллана.

GPU-ийн global memory нь их хэмжээний өгөгдөл хадгалах боломжтой боловч register болон shared memory-тэй харьцуулахад удаан. Энэхүү хэрэгжүүлэлтэд merge хийх өгөгдөл global memory дээрээс уншигдаж, global memory руу бичигдэж байгаа тул memory bandwidth нь CUDA хувилбарын гүйцэтгэлд хүчтэй нөлөөлнө.

CUDA bottom-up Merge Sort нь шат бүрт бүх merge segment-үүдийг зэрэг ажиллуулах боломжтой боловч шат бүрийн дараа дараагийн width рүү шилжихийн тулд kernel call дууссан байх шаардлагатай. Энэ нь implicit синхрончлол маягаар ажиллана.

8. Дүгнэлт

Энэхүү бие даалтын ажлаар Merge Sort алгоритмыг дараалсан, `std::thread`, OpenMP, CUDA гэсэн дөрвөн өөр хувилбараар хэрэгжүүлж, 10,000, 100,000, 1,000,000 элемент дээр гүйцэтгэлийг харьцуулан шинжилсэн.

Дараалсан хувилбар нь алгоритмын үндсэн ажиллагааг ойлгоход хамгийн энгийн бөгөөд baseline хэмжилт болгон ашиглагдсан. `std::thread` хувилбар нь C++ стандарт сангийн thread ашиглан зүүн болон баруун дэд массивуудыг зэрэг эрэмбэлэх боломжийг харуулсан. OpenMP хувилбар нь compiler directive ашиглан task-based parallelism хэрэгжүүлж, thread management-ийг OpenMP runtime-д даатгаснаараа кодын хувьд илүү цэвэр болсон. CUDA хувилбар нь GPU-ийн grid, block, thread бүтэц ашиглан bottom-up Merge Sort хэрэгжүүлсэн.

Туршилтын үр дүнгээс харахад жижиг хэмжээний өгөгдөл (10,000) дээр параллелчлалын overhead их нөлөөлж, зарим тохиолдолд дараалсан хувилбараас удаан ажилласан. Харин өгөгдлийн хэмжээ өсөхөд CPU-д суурилсан параллел хувилбаруудын давуу тал тодорхой болж, ялангуяа OpenMP болон `std::thread` хувилбарууд 3–5 дахин хурдан ажилласан.

Гэвч CUDA хувилбар энэхүү хэрэгжүүлэлтийн хувьд хамгийн удаан ажилласан. Үүний шалтгаан нь зөвхөн өгөгдөл дамжуулалт биш бөгөөд kernel execution хугацаа нийт хугацааны ихэнхийг эзэлж байсан. Merge Sort алгоритм нь олон удаагийн санах ойд хандах (memory access) шаардлагатай, мөн merge үе шат бүр нь бүрэн параллелчлагдах боломжгүй тул GPU архитектурын давуу талыг бүрэн ашиглаж чадаагүй.

Иймээс GPU ашиглах нь бүх тохиолдолд гүйцэтгэлийг сайжруулна гэж үзэх нь буруу бөгөөд алгоритмын онцлог, memory access хэв маяг, параллелчлах боломж зэрэг хүчин зүйлсийг харгалзан үзэх шаардлагатай. Merge Sort зэрэг divide-and-conquer алгоритмуудын хувьд CPU дээрх олон цөмт хэрэгжилт илүү үр ашигтай байж болохыг энэхүү туршилт харууллаа.

Номзүй

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Specification*, Version 5.2, 2021.
- [3] NVIDIA Corporation, *CUDA C++ Programming Guide*. NVIDIA Developer Documentation.
- [4] A. Williams, *C++ Concurrency in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2019.
- [5] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Boston, MA, USA: Addison-Wesley, 1998.